# Firmware Emulation With an Automated Skill Set

Firmware Research Team

May 21, 2026

## Contents

# 1 The Problem

Imagine receiving a firmware file one day, perhaps a `.bin`, `.img`, `.raucb`, or `.img.gz`. The first instinct is often: "We have the file, so let's just run it." That sounds reasonable. But firmware is not a normal `.exe` that we can double-click and wait for a window to appear.

Firmware is more like a sealed box containing a small slice of the device's world. In a router, it may include a compact operating system, network configuration, a web UI, system management daemons, an SSH server, a firewall, and a package manager. In a charging station, it may include charging-control services, certificates, backend communication, CAN bus handling, OCPP, Modbus, certificate management, an internal database, and a web management UI.

In other words, firmware is rarely just "one program". It is usually a small ecosystem.

The real question is: from that sealed box, how do we rebuild a virtual device that is realistic enough to interact with? Not only to see boot logs, but also to open the web interface, log in, exercise important functions, and understand exactly where an error comes from when a route returns a failure.

That is why I built this firmware emulation skill set.

# 2 What Is Firmware Emulation?

Firmware emulation means creating a simulated environment where the firmware believes it is running on real hardware. For example, if the firmware was built for a MIPS router, we need a suitable MIPS virtual machine; if the firmware is x86, we need an x86 environment. Beyond that, we need many matching pieces to wake the system up properly.

A simple way to picture full-system emulation is to think of the main layers we need to reconstruct:

- We need a virtual hardware layer that plays the role of CPU, RAM, disk, network card, serial port, and similar components. In this work, I use **QEMU**, a widely used tool that is capable enough for this job.
- **Kernel** - the component that wakes the system up. For emulation to work, the kernel must know how to mount the root filesystem, start the first process, and communicate with drivers.
- **Rootfs** - the disk-like filesystem that contains programs, libraries, configuration files, init scripts, and services.
- **Init system** - the first manager: which service starts first, which service starts later, and where logs go.
- **Services** - background programs such as web servers, SSH, configuration daemons, databases, and message buses.

One thing I misunderstood at first was this: booting does not mean successful emulation. Booting only means the virtual device has shown signs of life. The firmware can still sit there doing very little.

I divide emulation status into several layers:

1. **Booted**: the kernel runs, the rootfs is mounted, and init starts.
2. **Services are up**: the required daemons exist and listen on the expected ports.
3. **Host can reach it**: the service can be opened from the host through localhost or the configured route.
4. **Features are usable**: after authentication, if required, representative feature routes work without main-path errors such as `500`, `502`, `404`, or `400`. Hardware-specific features may still need to be scoped separately.

## 3 Why Firmware Emulation Is Harder Than It Looks

At a glance, firmware emulation may look like a matter of choosing the right tool-supported components, putting them in the right order, and running a script. In practice, it is partly that, but for people who are new to firmware research, the hard part is usually not knowing what the firmware actually needs, or choosing one important component incorrectly. AI can help a lot with debugging these problems, but without a skill that knows what questions to ask, it is very easy to get lost in a sea of logs and QEMU options.

Here are several problems I ran into when trying to emulate firmware through manual prompting:

- **AI debugging may not go deep enough, leading to the wrong CPU architecture and repeated dead loops**: MIPS firmware cannot run like x86 firmware. ARM, ARM64, MIPS big-endian, MIPS little-endian, and x86_64 are different worlds.
- **A rootfs exists, but the kernel is missing**: it is like having a disk but nobody to start the machine.
- **The kernel boots, but init does not come up**: the system wakes up and then just stands still.
- **Networking is wired incorrectly**: the guest has its own network, the host has its own network, port forwarding appears to be open, but the service remains silent.
- **The network card model is wrong**: for example, QEMU may attach an `e1000` card, while the firmware only has a `pcnet32` driver. The result is that the guest does not know how to talk to that device.
- **Small missing pieces such as daemons, databases, or configuration files**: these gaps cause services and backend APIs to return errors when we interact with them.
- **Real hardware is missing**: this is often the hardest problem. Vendor-specific hardware can be central to how firmware operates, but it may be impossible or very difficult to emulate directly, such as CAN bus, modem, TPM, NVRAM, serial device, storage partition, sensor, or fieldbus service dependencies.

So when someone says “the firmware has been emulated”, the next question should be: emulated to what level? Boot log? Shell? Web service? Login? Or actual usable features?

## 4 The Business Problem: What Breaks When This Is Manual?

In research reports, environment setup time is often invisible, but it consumes a lot of real work. One firmware image can take hours, sometimes days, just to find the right way to boot. When documentation is limited, the time cost grows because every step becomes a cycle of guessing, trying, reading logs, changing QEMU options, patching configuration, and trying again.

The problem is not only time. There are three larger business issues:

- **Results are hard to standardize**: each engineer may interpret “emulated successfully” differently.
- **Results are hard to reproduce**: without clear records of the kernel, QEMU command, patches, port mappings, and route tests, another person cannot easily run the same setup again.
- **Results are hard to audit**: when we conclude that a service works or a route fails, we need to know whether the evidence came from real runtime behavior or only from static analysis.

The skill set is designed to force the process into an evidence-based pipeline: what was done, what was observed, what was changed, what is still missing, what the final result is, and what scope that result actually covers.

## 5 The Core Idea of the Skill Set

The central idea is simple: before we can say a firmware image has been emulated successfully, we need to know what the firmware is, which architecture it targets, which kernel or rootfs it needs, which services must come up, whether the APIs used to operate those services actually work or only return errors, and where a failure belongs if something breaks. The cause may be the web server, reverse proxy, backend daemon, database, configuration, or a hardware dependency that does not exist in the lab environment.

In other words, the skill set turns a short request such as "please emulate this firmware" into a group of concrete actions. Each action has a clear input, a clear output, and an artifact that later steps can read.

There are four important design principles.

1. **Split the large problem into smaller roles.** Firmware emulation is a layered problem: firmware format, CPU architecture, kernel, init, filesystem, network, service, UI, authentication, backend, logs, dependencies, and emulated hardware. If everything is placed into a single skill, the workflow easily becomes a long script that is hard to control. The skill set separates the work into smaller roles so each skill can do its own part well.
2. **Follow evidence, not intuition.** Every conclusion needs an artifact behind it: which file was read, which service ran, which port listened, which route was tested, which error appeared, and which log proves it. If runtime evidence is missing, the result must be called a hypothesis, not a success.
3. **Keep the claim honest.** The claim should match the level actually reached. For example, if only a userland service can be run, the result must be described as userland or service emulation, not full-device emulation.
4. **Use real usability as the metric.** The highest goal is that a user can meaningfully interact with the firmware's important functions. Errors should not be ignored when they sit on the main usage path.

At a high level, the skill set works like this. Each JSON artifact has common metadata: who created it, when it was created, which input it was based on, whether there are warnings or errors, what sensitivity level applies, and what evidence level currently exists. To keep the explanation readable, the **Output** sections below do not list every field. They describe the most important groups of information to remember.

- `firmware-start`
  - **Task:** acts as the entry point for the whole workflow. It is used when the user only has a firmware file or an extracted directory and does not yet know which skill should run next.
  - **Input:** firmware path or extracted directory, authorization notes if available, model/version hints if available, goals such as full-system boot, service readiness, workload/API/IPC validation, WBM/UI when the firmware has an interface, authentication if required, and an output workspace if the user specifies one. In plain language, the input is: which firmware we have, what we are allowed to do with it, and how far we want to get.
  - **Output:** `starter_request.json`, `next_skill_decision.json`, and `blockers.json` if information is missing. In short, `starter_request.json` is the initial request ticket: which firmware, what authorization, what goal, where the workspace is, and which skill should receive the handoff. `next_skill_decision.json` explains which skill should run next and why. `blockers.json` records what is missing or unsafe to continue with.
  - **Goal:** turn a short prompt into a structured request, preserve the success gate from the beginning, and clarify success criteria for the firmware type.
- `firmware-artifact-contract`
  - **Task:** defines the common rules for every artifact in the pipeline. This skill does not reverse engineer, debug, or run firmware; it checks how evidence is recorded.

  - **Input:** any JSON artifact, JSONL observation stream, finding record, discovery artifact, or candidate report produced by other skills.
  - **Output:** validation status for the artifact, warnings that need attention, and blockers if the artifact is not reliable enough. For status files such as `emulation_success.json`, this skill keeps the gates separate: boot, service, host reachability, interface/API, authentication if present, workload/feature, and dependency scope.
  - **Goal:** ensure conclusions have provenance, schema, sensitivity labels, warnings/errors, and do not collapse different states into one vague "pass". Booted, service ready, host reachable, interface/API usable, auth handled, feature observed, and dependency scoped are different levels and must be recorded separately for the firmware context.

- `firmware-intake-normalization`
  - **Task:** normalize the firmware before deeper analysis. This step answers basic questions: what file is this, what are its hashes, how was it extracted, where is the rootfs, and which seed services or binaries are worth noticing.
  - **Input:** firmware artifact, extraction metadata, and hashes. The input can be the original raw file or an existing extraction result, but it must contain enough information to trace the source.
  - **Output:** `firmware_manifest.json`, `extraction_summary.json`, and a binary seed list. `firmware_manifest.json` is the firmware's identity card: hashes, format, model/version hints, predicted architecture, and primary rootfs. `extraction_summary.json` is the extraction log: which tools were used, which partitions/filesystems were found, where the rootfs is, and which errors occurred. The binary seed list highlights a few binaries or services to inspect first.
  - **Goal:** create a concrete identity for the firmware so later steps are not working on an ambiguous folder. If the primary rootfs is unknown, extraction failed, or there are unexplained partitions, the skill must record a blocker instead of guessing.

- `firmware-binary-service-inventory`
  - **Task:** inventory binaries and services inside the identified rootfs. It looks for executables, scripts, interpreters, shared libraries, init paths, daemons, run permissions, port hints, and service candidates.
  - **Input:** `firmware_manifest.json` and a rootfs candidate. In simple terms, this is the point where the workflow already knows "what firmware this file tree belongs to" and starts asking "which programs inside it may need to run".
  - **Output:** `binary_inventory.json`, `service_inventory.json`, and `service_graph.json`. These three files answer three questions: which programs exist, which programs look like services that need to run, and how those services relate to init scripts, configuration, ports, libraries, or dependencies.
  - **Goal:** create the first static map of services that may need emulation, so the workflow does not have to search file by file based on intuition.

- `firmware-attack-surface-graph`
  - **Task:** connect services with input vectors, parsers, sinks, privileges, and authentication boundaries into an evidence-backed graph. Although the name sounds security-oriented, it is very useful for emulation because it shows which services matter and which interaction paths must be validated once runtime is available.
  - **Input:** `binary_inventory.json`, `service_inventory.json`, and `service_graph.json`.
  - **Output:** `attack_surface_graph.json` and `target_selection.json`. The graph shows which service receives input from where, which parser or boundary it crosses, and whether the evidence is strong or weak. The target selection file chooses the services or interaction paths to prioritize first, with reasons.

  - **Goal:** prioritize the right targets: which service is exposed to the network, which service handles input from API/IPC/CLI/update/config, which service runs with high privilege, and which service still lacks reachability evidence.

- `firmware-input-path-discovery`
  - **Task:** discover input paths into the selected service: HTTP routes, CLI, IPC, configuration files, environment variables, update packages, network messages, or other control paths.
  - **Input:** `target_selection.json`, `attack_surface_graph.json`, and the selected binary/service.
  - **Output:** `input_vectors.json` and `input_path_chains.json`. The first file lists controllable input sources such as HTTP, IPC, CLI, config, or update package. The second describes how that input travels before it reaches a service, handler, parser, or observable effect.
  - **Goal:** understand how the service can be controlled, so during emulation we know which workload to create, which message to send, which file/config to prepare, which endpoint to call, and which inputs remain static guesses without runtime proof.

- `firmware-discovery-orchestrator`
  - **Task:** coordinate the whole pipeline based on current evidence. It does not replace the work of other skills; it reads artifacts, identifies what is missing, and chooses the next skill most likely to increase evidence.
  - **Input:** artifact index, `firmware_manifest.json`, `hypothesis_ledger.json`, and current evidence levels.
  - **Output:** `discovery_plan.json`, `target_selection.json`, `hypothesis_ledger.json`, `next_skill_decision.json`, and `blockers.json`. Practically, this is the coordination map: what is currently known, what evidence is missing, which hypotheses are open, which skill should run next, and which blocker is stopping progress.
  - **Goal:** keep the workflow from drifting. If the goal is full-system, full service readiness, usable workload, observable API/IPC, or UI/auth when the firmware has those layers, the orchestrator must preserve the `emulation_success_gate` and continue routing to state discovery, service emulation, runtime observation, or debugging until there is enough evidence or a clear blocker.

- `firmware-service-state-discovery`
  - **Task:** map service state, authentication, dependencies, and reachability before deep runtime work. For each service, it asks which files, configs, helper daemons, data directories, credentials or initial states, ports/sockets, and hardware dependencies or simulators are required. If the firmware has a web UI, it also records browser entrypoints, static assets, API base URL, WebSocket, reverse proxy, login endpoint, and session/token storage.
  - **Input:** `service_inventory.json`, `attack_surface_graph.json`, and `input_vectors.json`.
  - **Output:** `service_state_map.json`, `reachability_blockers.json`, and `ui_access_requirements.json`. `service_state_map.json` is the condition map for making a service run: required files, configs, helper daemons, ports/sockets, auth, and hardware/simulator dependencies. `reachability_blockers.json` records why the service cannot yet be reached. `ui_access_requirements.json` is used only when the firmware has an interface, to record entrypoints, API/WebSocket paths, reverse proxy behavior, and feature routes.
  - **Goal:** know in advance what runtime needs in order for the service to become genuinely usable, from database, certificate, message broker, storage partition, CAN/serial/GPIO/modem/TPM to simulator. This step gives every runtime failure a place to be traced back to, not only web-interface failures.

- `firmware-service-emulation`

  - **Task:** build the runtime environment for the selected service or firmware lane. It checks files, libraries, device nodes, ports, environment variables, init state, backend listeners, and dependencies before calling the runtime ready.
  - **Input:** `service_inventory.json`, `service_state_map.json`, and `hypothesis_ledger.json`.
  - **Output:** `runtime_readiness.json`, `reachable_services.json`, `ui_access_status.json`, `emulation_success.json`, or `emulation_blocker.json`. These files state which runtime lane was used, which services are running, whether the host can reach the required entrypoint, what state the UI/API is in if present, and which gate passed or failed. If it fails, the blocker must name the broken gate and suggest the next debugging direction.
  - **Goal:** bring the firmware or service into a verifiable runtime and clearly record whether the lane is `qemu-system`, `qemu-user`, chroot, container, proxy, or static-only. The skill does not call the work successful if the main process is alive but required dependencies are dead, the host cannot reach the required entrypoint, the representative workload cannot run, or auth/interface behavior is only partially working and not scoped.

- `firmware-runtime-observation`
  - **Task:** observe real behavior once the runtime can run. This skill uses logs, tracing, tcpdump, Frida, bpftrace, debugger-safe traces, or browser-level observation to prove that a real workload passed through the system.
  - **Input:** `runtime_readiness.json`, hypothesis, workload, and expected observation. A workload can be an API call, IPC message, CLI command, config load, simulated update package, web screen, or a read-only post-auth feature when the firmware requires authentication.
  - **Output:** `runtime_observation.json`, `ui_runtime_observation.json`, `observed_path_chains.json`, `observed_sink_hits.json`, and `debug_transcript_index.json`. These files answer: which workload ran, what was expected, what was actually observed, where the log/trace evidence lives, and which input-to-effect path was truly observed. If there is a UI or API, `ui_runtime_observation.json` records the route/workload matrix at a redacted level.
  - **Goal:** turn "I think it runs" into "I observed this execution path". For firmware with an interface or authentication, the skill checks the path a real user would take and does not record secrets, tokens, or cookies. For headless firmware, it focuses on API, IPC, CLI, daemon state, packet flow, or log/runtime effects.

- `firmware-debugging`
  - **Task:** dig into root cause when there is a crash, runtime error, sink hit, or concrete root-cause question. Examples include a service dying at startup, a backend not opening a port, an endpoint returning `500`, an API returning `404`, IPC not responding, WebSocket failing to connect, a binary requiring a missing device, or an init script stopping halfway.
  - **Input:** crash or suspicious effect, target process, `debug_plan.json`, and hypothesis.
  - **Output:** `debug_transcript_index.json`, crash-analysis handoff, or exploitability-modeling handoff if needed. The index is not a raw debug dump; it is a summary of which process was debugged, which attach/probe method was used, what crash/effect was seen, where the evidence is, whether runtime was modified, and what the temporary root cause appears to be.
  - **Goal:** find root cause in a controlled way using GDB, gdbserver, QEMU gdbstub, guest breakpoints, host-side QEMU attach, logs, or appropriate traces. It also records which actions modified runtime behavior so readers do not confuse original firmware behavior with behavior after shim or patch intervention.

- `firmware-memory-layer`

- **Task:** turn validated experience into reusable patterns for future firmware cases. This skill must not be used to silently change core rules or store unproven assumptions.
- **Input:** validated artifact, pattern draft, source evidence, and artifact sensitivity label.
- **Output:** `memory_suggestions.json`, `memory_draft.md`, `promotion_decision.json`, or reusable patterns. In simple terms, this is where the workflow records which pattern can be reused, which evidence proves it, whether it is ready for promotion, or whether it still needs more validation.
- **Goal:** make the workflow smarter after each case while preserving discipline: every pattern needs a verification date, source evidence, no secrets, and must not replace validation of the current artifact.

The workflow can be pictured like this: `firmware-start` records the goal. `firmware-discovery-orchestrator` checks what evidence currently exists. If firmware identity is missing, it routes to intake. If the service list is missing, it routes to inventory. If runtime state, auth, dependencies, or workload are not understood, it routes to state discovery. If enough runtime conditions exist, it routes to service emulation. Once runtime is up, runtime observation validates it with a suitable workload: API, IPC, CLI, packet flow, log effect, or browser path when the firmware has an interface. If an error appears, debugging loops back to find the cause. After every round, new artifacts are written, the orchestrator reads them again, and the next decision is made. The loop stops when the success gate is reached or when a blocker is clear enough to explain exactly why the gate cannot be reached yet.

## 6 How Beginners Can Use It Effectively

Use one one-shot prompt for each firmware image. State the goal clearly; do not only say “boot this firmware”. It is also useful to enable the **superpowers** plugin and use `/goal` to improve the output target, so you do not need to prompt back and forth too many times.

```
Use firmware-start to one-shot full-system emulate firmware
/path/to/A.bin, create start/stop/restart scripts,
and record clear blockers if the success gate is not reached.
```

## 7 Experiment Results

In the extended test scenario, the skill set was run against **12 firmware samples**: 10 OpenWrt samples across different targets and 2 **CHARX SEC-3100** samples, versions **1.7.1** and **1.9.0**. The important point is not only the number of samples, but also the diversity of CPU architectures and firmware packaging styles. The skill set did not only solve an easy “OpenWrt x86 runs well” case. It had to move across several different worlds: full disk image, rootfs-only image, sysupgrade image, SD card image, and an industrial-device RAUC bundle.

| Firmware group | Count | CPU architecture | Input type | Result |
|---|---|---|---|---|
| OpenWrt x86_64 generic | 1 | x86_64 | Full disk image | Full-system boot, service ready, host reachable, route matrix OK |
| OpenWrt ARM/ARM64 boards | 6 | ARMv7, ARM64/AArch64, Cortex-A9, Filogic/IPQ-class | Factory image, SD card image, sysupgrade image | Full-system boot succeeded after choosing the right machine, NIC, and storage profile |
| OpenWrt MIPS targets | 3 | MIPS little-endian and MIPS big-endian, including Malta BE | Sysupgrade image, rootfs-only image | Full-system boot succeeded while handling endian differences, external kernel, and NIC driver requirements |
| CHARX SEC-3100 | 2 | ARM Cortex-A7, i.MX6UL-class embedded platform | RAUC bundle, rootfs/kernel/DTB | Full-system boot through kernel/init, service stack and WBM/API workload reached the success gate |

Overall, the test set covers at least four major architecture families: **x86_64**, **ARMv7/ARM Cortex-A**, **ARM64/AArch64**, and **MIPS**, including both little-endian and big-endian variants. This is the most valuable point of the skill set: the workflow is not locked to one CPU or one image format. When moving from OpenWrt to CHARX SEC-3100, the problem also changes from router firmware to industrial-device firmware with WBM, auxiliary services, hardware dependencies, and a RAUC update bundle. Keeping the same success gate across these very different groups shows that the skill set can act as a general working framework, not only as a one-off QEMU script written for a single model.